

<https://grafana.com/docs/grafana/latest/fundamentals/intro-to-prometheus/>

What is Prometheus? (The Health Inspector)

Imagine you're running a massive automated factory (your applications and servers). You need a system that constantly checks the health and performance of every machine.

Prometheus is an open-source system monitoring and alerting toolkit. It acts as a dedicated "**Health Inspector**" for your entire infrastructure.

Its main job is to collect **time series data**—which are just **numerical measurements (metrics)** recorded with a specific **timestamp**.

How Prometheus Works (The Core Mechanism)

Prometheus uses a simple, reliable "**pull**" **model** to collect data:

1. Metrics (The Measurements)

- These are the numerical measurements you care about.
- **Example:** For a web server, a metric could be the **request count per second**, or the **average time it takes to process a request**.
- Metrics help you understand *why* your application is behaving in a certain way (e.g., if request time is high, check if request count is also high).

2. The Pull Model (Scraping)

- Instead of waiting for applications to send data (push), the **Prometheus Server** actively goes out and "**scrapes**" (pulls) the metrics from monitored targets over **HTTP** at regular intervals.
- The targets are discovered via configuration or service discovery.

3. Labels (The Context)

- Metrics are stored with **labels** (key-value pairs) that add context.
- **Example:** You have a metric called `http_requests_total`. A label might be `endpoint="/login"`, allowing you to query requests only for the login page. This gives the data **multi-dimensionality**.

4. PromQL (The Query Language)

- Prometheus has its own powerful language called **PromQL**.
- This allows you to select, filter, and aggregate your multi-dimensional time series data (e.g., "Show me the average request time across all servers in the 'US East' region over the last 5 minutes").

Key Components (The Team)

Prometheus isn't just one tool; it's an ecosystem of connected components:

- **Prometheus Server:** The core brain. It **scrapes** the data, **stores** it locally, and runs **rules** (for recording or alerting).
- **Exporters:** These are small helper programs that take metrics from systems not built for Prometheus (like databases or third-party services) and convert them into the format Prometheus can scrape.
- **Client Libraries:** Code you use to instrument your *own* application (e.g., a Python or Go app) so it can expose its metrics endpoint for Prometheus to scrape.
- **Push Gateway:** A required intermediary for **short-lived jobs** (like a batch script) that don't live long enough to wait for Prometheus to scrape them. They push their single data point here.
- **Alertmanager:** A separate component that **handles alerts** generated by the Prometheus server, deciding how to route them (e.g., to email, Slack, or PagerDuty).

When is Prometheus the Right Fit?

- **During Outages:** It's designed for **reliability**. Each Prometheus server is **standalone** (no reliance on distributed storage). You can rely on it to tell you what's wrong even if your main database or network storage is failing.
- **Microservices:** Its support for **multi-dimensional data** (labels) makes it excellent for monitoring complex, dynamic architectures where services are constantly spinning up and down.

When is Prometheus NOT a Good Fit?

- **Billing/100% Accuracy:** It prioritizes reliability and availability over perfect data completeness. If you need **100% guaranteed accuracy** for things like per-request billing, you should use a dedicated transactional system. Prometheus is best for **monitoring** and **diagnostics**.

What is Grafana? (The Visualization Hub)

Imagine you have many different data streams (logs, server stats, sales figures) coming from separate locations (Prometheus, MySQL, AWS). You need a central screen to see and understand all of it at once.

Grafana is a free, open-source tool used for creating dynamic, interactive dashboards to visualize time-series data from multiple sources.

It acts as the **"Control Panel"** or **"Unified Monitoring Dashboard"** for your entire system's health.

Key Features (Why Everyone Uses It)

1. Universal Data Connectivity (The Hub)

- **Multiple Sources:** Grafana doesn't store data itself; it connects to virtually any data source you already use.
- **Examples:** It links directly to popular monitoring systems like **Prometheus** and **Elasticsearch**, and traditional databases like **MySQL** and **PostgreSQL**. This gives you a single pane of glass for all your metrics.

2. Interactive Dashboards

- **Real-Time View:** Allows users to build dynamic, real-time dashboards to monitor system performance and application health.
- **Customizable:** Uses a simple drag-and-drop interface to add, remove, and rearrange **panels** (the individual graphs or visualizations).

3. Customizable Visualizations

- **Variety:** Provides numerous ways to display data, from simple **line graphs** and **bar charts** to complex **heatmaps** and **gauge displays**.
- **Advanced Querying:** Allows you to filter, aggregate, and transform your data *before* it's displayed, ensuring the visualization is meaningful.

4. Alerting and Notifications

- **Active Monitoring:** Users can define **alert rules** based on specific thresholds (e.g., "Alert me if CPU usage is >90% for five minutes").
- **Dispatch:** Alerts can be sent immediately to various channels like email, **Slack**, or PagerDuty, enabling rapid incident response.

Common Use Cases

Grafana is the visualization layer used for almost any monitoring or analytics task:

- **Infrastructure Monitoring:** Tracking server health (CPU, memory, network traffic) in real-time.
- **Application Performance Monitoring (APM):** Visualizing application response times, error rates, and throughput.
- **Business Metrics:** Tracking key performance indicators (KPIs) like sales volume, website traffic, or customer engagement.
- **Log and Event Monitoring:** Combining logs (from tools like Elasticsearch or Loki) with performance metrics on one screen for easier troubleshooting.

Setting It Up (Simplified)

1. **Install Grafana:** Install the software on your system.
2. **Add Data Sources:** Log in and configure the connection details for your data tools (e.g., point it to your Prometheus server).
3. **Create Dashboards:** Use the web interface to pull metrics from your sources and create custom visual panels.
4. **Set Up Alerts:** Define rules within the panels to notify teams when a metric crosses a critical threshold.

Grafana is often the final piece in a monitoring puzzle, turning raw data into clear, actionable insights.

Grafana's architecture is quite straightforward because it primarily functions as a **visualization layer** and relies on **external data sources** for its information.

Here is a breakdown of the components and architecture of Grafana.

Grafana Components and Architecture

Grafana is best understood as a sophisticated, independent web application that connects, queries, and visualizes data, but **does not store the time-series data itself**.

1. The Core Application

The Grafana application runs as a single server or service.

- **Web Server:** This is the core component that serves the **HTML, CSS, and JavaScript** that makes up the **Grafana Web Interface**. Users interact with the dashboards through this server (usually accessed via port 3000 by default).
- **Database (Internal):** Grafana uses a small internal database (often SQLite, MySQL, or PostgreSQL) **not** to store time-series metrics, but to store metadata about its own configuration:
 - User accounts and permissions.
 - Dashboard definitions (the layout, panels, and queries).
 - Alert rules and notification channels.
- **Backend Engine:** This component handles the complex tasks:
 - Processing and executing user **queries** against the external data sources.
 - Running **alert checks** against the query results at set intervals.

2. External Components (The Data Flow)

The power of Grafana comes from its ability to integrate with the outside world.

Component	Role	Function in Grafana
Data Sources	The external systems that hold the actual metric, log, or trace data.	Grafana uses plugins to connect to these sources (e.g., Prometheus, Elasticsearch, AWS CloudWatch, MySQL) and sends them queries.
Dashboards	The visualization canvases created by the user.	A single dashboard can contain multiple Panels that pull data from <i>different</i> Data Sources.
Panels	The individual visualization units (e.g., a line graph, a gauge, a table).	Each Panel executes a specific Query against a selected Data Source to render the visual data.
Alerting System	The framework that manages notification rules.	Grafana constantly queries the Data Source, compares the result to the defined threshold , and uses Notification Channels (like Slack or email) to send alerts if the threshold is breached.
Plugins	Extendable modules.	Allow Grafana to connect to new/non-standard data sources or provide unique visualization types.

3. Architecture Summary

In essence, the flow is:

1. A user loads a **Dashboard**.
2. The Grafana Backend Engine executes the **Queries** defined in the Dashboard's **Panels**.
3. The queries are sent through the appropriate **Data Source Plugin** to the external storage system (e.g., Prometheus).
4. The external system returns the raw time-series data.

5. Grafana renders this data into the specified **visualization** on the user's screen.

This architecture ensures Grafana is **lightweight** and **flexible**, as it is not burdened with storing petabytes of time-series data itself.